

计算机网络面试题

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

🔴 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**

嵌入式面试中网络知识越来越重要(IoT/车联网/工业以太网)。本章覆盖TCP/IP协议栈、Socket编程、HTTP/DNS等高频考点。

侧重嵌入式相关内容(IwIP/MQTT/CoAP)，配秒懂和代码。

★ 网络核心概念图解

◆ TCP/IP四层模型

- **应用层**(HTTP/MQTT)是你写的信
- **传输层**(TCP/UDP)是快递公司,保证送达或只管发
- **网络层**(IP)是路由规划走哪条路
- **链路层**(以太网/WiFi)是具体的卡车和公路

OSI七层	TCP/IP四层	典型协议
应用/表示/会话	应用层	HTTP/MQTT/DNS/DHCP
传输层	传输层	TCP/UDP
网络层	网络层	IP/ICMP/ARP
数据链路/物理	链路层	以太网/WiFi/PPP

// MQTT协议

数据封装：
应用数据 → [TCP头|数据] → [IP头|TCP头|数据] → [帧头|IP头|TCP头|数据|帧尾]
段(Segment) 包(Packet) 帧(Frame)

★ 网络协议栈与嵌入式全景对比

OSI七层 vs TCP/IP四层 vs 嵌入式常用：			
OSI七层	TCP/IP四层	嵌入式常用	
应用层	应用层	MQTT/CoAP/HTTP/cJSON	// MQTT协议
表示层		Protobuf/JSON	
会话层		TLS/SSL(mbedtls)	
传输层	传输层	TCP/UDP	

网络层	网络层	IP/ICMP/ARP
数据链路层	网络接口层	以太网/WiFi/BLE
物理层		PHY/RJ45/天线

嵌入式网络方案选型：

方案	协议栈	适用MCU	典型芯片
自带协议栈	w5500内部	任意MCU	w5500
lwIP	软件协议栈	≥64KB RAM	STM32+PHY
AT指令	模组内部	任意MCU	ESP8266
Linux	内核协议栈	Cortex-A	i.MX6

一、TCP/UDP基础（Q1~Q20）

Q1: TCP和UDP的区别？

 **秒懂：** TCP是电话(先拨号连接→可靠通话→挂断)，UDP是寄信(写好地址直接扔邮筒，不保证送达)。TCP可靠但慢，UDP快但可能丢。嵌入式中控制命令用TCP，实时音视频用UDP。

	TCP	UDP
连接	面向连接(三次握手)	无连接
可靠	可靠(重传/排序/流控)	不可靠(尽力而为)
有序	保证顺序	不保证顺序
速度	慢(有开销)	快★
头部	20字节	8字节
传输	字节流	数据报(有边界)
广播	不支持	支持★
适合	文件传输/Web/OTA	DNS/NTP/视频/MQTT(QoS0)

嵌入式选择：资源受限的MCU用UDP更轻量(如CoAP)；需要可靠的用TCP(如OTA固件下载)。

 **面试追问：** OSI七层和TCP/IP四层怎么对应？每层的代表协议？

 **嵌入式建议：** 嵌入式工程师重点掌握:传输层(TCP/UDP)+应用层(MQTT/HTTP)+物理层(以太网/WiFi/蓝牙)。

🚩 OSI七层 vs TCP/IP四层 对比

OSI七层	TCP/IP四层	功能	典型协议/设备
应用层	应用层	用户接口/数据表示	HTTP/MQTT/CoAP
表示层	↑	加密/压缩/编码	SSL/TLS/JSON
会话层	↑	会话管理	-
传输层	传输层	端到端可靠传输	TCP/UDP
网络层	网际层	路由/寻址	IP/ICMP/ARP
数据链路层	网络接口层	帧封装/MAC寻址	Ethernet/WiFi
物理层	↑	比特流传输	网线/光纤/无线

Q2: TCP三次握手?

💡 秒懂：三次握手建立连接：①客户端发SYN ②服务器回SYN+ACK ③客户端发ACK。三次的目的是确认双方都能收发——少于三次无法确认，多于三次浪费。



为什么是三次而不是两次？

两次：服务器收到SYN就确认连接

问题：如果是一个"历史SYN"(网络延迟很久才到达)

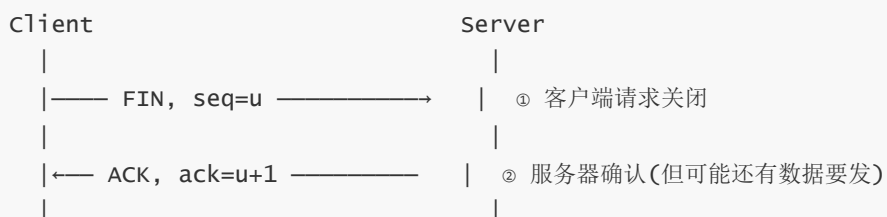
→ 服务器建立了错误的连接 → 浪费资源

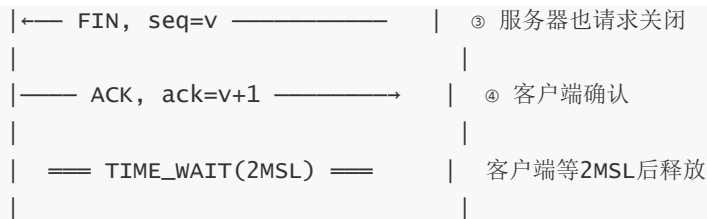
三次：客户端确认ACK → 证明这个SYN是"新鲜的"

Q3: TCP四次挥手?

💡 秒懂：四次挥手关闭连接：①主动方发FIN ②被动方回ACK(半关闭) ③被动方发FIN ④主动方回ACK。比建立多一次是因为被动方可能还有数据没发完，需要分两步关闭。

四次挥手的时序和TIME_WAIT状态是网络面试必考内容：





为什么要TIME_WAIT?

- ① 如果最后的ACK丢了,服务器会重发FIN → 客户端能重发ACK
- ② 确保旧连接的数据包在网络中消失(不影响新连接)

MSL = Maximum Segment Lifetime $\approx$ 2分钟

💡 **面试追问:** 三次握手每步做了什么? 为什么不是两次? SYN Flood攻击原理?

🔧 **嵌入式建议:** 嵌入式TCP编程必须处理:连接超时/重连机制/心跳保活/半开连接检测。

Q4: TCP的可靠性如何保证?

💡 **秒懂:** TCP可靠性靠: 序列号(保证顺序)、确认应答(保证到达)、超时重传(丢了再发)、校验和(保证完整)、流量控制(不溢出)、拥塞控制(不堵网)。六大机制缺一不可。

TCP通过多种机制保证数据可靠传输, 面试中需要能列举核心机制:

机制	作用
序列号/确认号	保证数据有序、检测丢包
超时重传	丢包后自动重发
滑动窗口	流量控制, 防止接收方溢出
拥塞控制	防止网络拥塞崩溃
校验和	检测数据损坏
快速重传	3个重复ACK立即重传

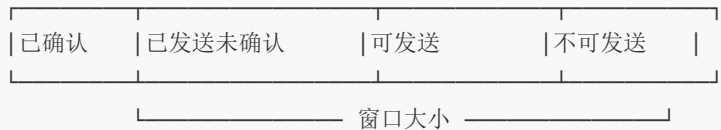


Q5: TCP滑动窗口机制?

💡 **秒懂:** 滑动窗口控制发送方一次能发多少未确认的数据。接收方通过窗口大小告诉发送方'我还能收N字节'。窗口=0时暂停发送。实现了流量控制——发送速度适配接收能力。

滑动窗口实现流量控制, 限制发送速率使接收方不溢出:

发送窗口(发送方维护):



接收窗口(接收方通告):

TCP头部 window 字段 → 告诉发送方"我还能接收多少数据"

窗口为0: 发送方停止发送(零窗口探测定时器)

窗口缩小: 出现Silly window Syndrome(小段数据) → Nagle算法优化

💡 **面试追问:** TCP和UDP各适合什么场景? MQTT用TCP还是UDP?视频流呢?

🔧 **嵌入式建议:** IoT设备首选MQTT(基于TCP,轻量);传感器突发上报用UDP;音视频用RTP/UDP。

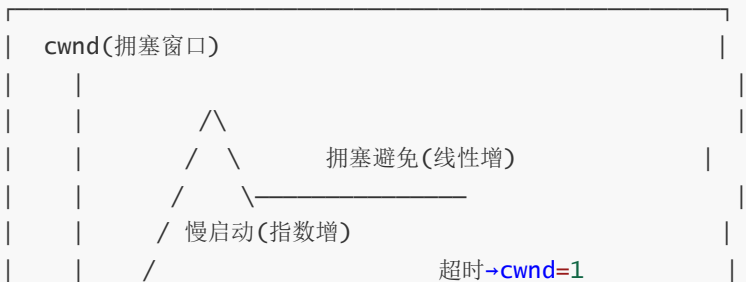
📊 TCP vs UDP 对比表

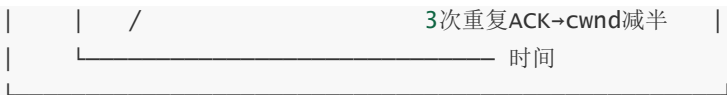
特性	TCP	UDP
连接	面向连接(三次握手)	无连接
可靠性	可靠(确认重传)	不可靠(尽力交付)
顺序	有序(序列号)	无序
流量控制	滑动窗口	无
拥塞控制	有(慢启动等)	无
头部开销	20字节	8字节
传输效率	较低(确认机制)	高
嵌入式场景	HTTP/MQTT/OTA升级	DNS/NTP/音视频流/IoT上报

Q6: TCP拥塞控制算法?

💡 **秒懂:** 慢启动(指数增长)→拥塞避免(线性增长)→拥塞发生(快重传+快恢复或超时)→降速重来。像开车: 先踩油门加速→匀速巡航→遇到堵车就减速→路况好了再加速。

TCP拥塞控制防止网络过载, 是面试高频考点:





四个阶段：

1. 慢启动：cwnd从1开始, 每个RTT翻倍(指数增长)
2. 拥塞避免：cwnd超过sssthresh后, 每个RTT+1(线性增长)
3. 快速重传：3个重复ACK → 立即重传 → cwnd减半
4. 快速恢复：不回到慢启动, 从cwnd/2开始拥塞避免

Q7: TCP的TIME_WAIT状态？为什么需要？

💡 秒懂：主动关闭方经历TIME_WAIT等待2MSL(约60秒)。原因：①确保最后一个ACK能到达对方 ②等待残余报文在网络中消失(防止新连接收到旧数据)。服务器重启时SO_REUSEADDR跳过等待。

💡 面试高频 | 服务端开发/嵌入式网络调试高频 | GitHub面经高赞题

TIME_WAIT是TCP四次挥手后主动关闭方的状态，持续2MSL(通常60秒)：

为什么需要TIME_WAIT？

1. 确保最后一个ACK能到达对方(如果丢失，对方会重发FIN)
2. 让网络中残留的数据包消亡(防止新连接收到旧数据)

嵌入式中的问题：大量TIME_WAIT占用端口和内存

```
# 查看TIME_WAIT数量
ss -s | grep TIME-WAIT

# 解决方案
# 1. SO_REUSEADDR(允许复用TIME_WAIT的端口)
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
# 2. 内核参数调节
echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse
echo 30 > /proc/sys/net/ipv4/tcp_fin_timeout
```

Q8: TCP的Nagle算法和延迟ACK？

💡 秒懂：Nagle算法：小数据包凑大了再发(减少小包数量)。延迟ACK：等一会再发ACK(可能捎带数据一起发)。两者同时开启可能导致延迟增大。嵌入式低延迟场景用TCP_NODELAY关闭Nagle。

Nagle算法减少小包发送，延迟ACK减少ACK数量：

Nagle算法规则：

如果有已发送未确认的数据 → 缓存小数据 → 等ACK到后一起发
如果没有未确认数据 → 立即发送

延迟ACK：

收到数据不立即回ACK → 等待一段时间(~40ms)或等待可以捎带ACK

二者冲突：Nagle等ACK才发数据，ACK延迟等数据才发ACK → 死锁40ms！

解决：关闭Nagle → TCP_NODELAY

```
// 嵌入式实时通信：关闭Nagle算法
int flag = 1;
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(flag));
```

Q9: TCP Keep-Alive机制?

 **秒懂：** TCP Keep-Alive定期发探测包检测连接是否存活。默认2小时才开始探测——太慢。嵌入式中通常在应用层自己实现心跳包(间隔几秒到几十秒)，更灵活可控。

TCP长连接保活检测对方是否存活：

```
int keepalive = 1;
int idle = 60;    // 60秒无数据开始探测
int interval = 10; // 每10秒探测一次
int count = 3;    // 3次无响应认为断开

setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &keepalive, sizeof(keepalive));
setsockopt(fd, IPPROTO_TCP, TCP_KEEPIRL, &idle, sizeof(idle));
setsockopt(fd, IPPROTO_TCP, TCP_KEEPIRLVL, &interval, sizeof(interval));
setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &count, sizeof(count));
```

应用层心跳 vs TCP Keep-Alive:

- TCP Keep-Alive：系统级，检测物理连接(默认2小时太久)
- 应用层心跳：更灵活，可检测应用层存活

Q10: TCP半连接队列和全连接队列?

 **面试高频** | 服务端开发高频 | 追问SYN Flood

 **秒懂：** 半连接队列存SYN_RECV状态(收到SYN但没完成握手)，全连接队列存ESTABLISHED状态(已完成握手等accept)。队列满时新连接被丢弃。listen的backlog参数控制全连接队列大小。

服务器端accept之前的两个队列是面试常考：

```
客户端SYN到达 → 半连接队列(SYN Queue)
                ↓ 完成三次握手
                全连接队列(Accept Queue) → accept()取出
```

半连接队列满：SYN Flood攻击 → 解决：SYN Cookie

全连接队列满：服务器accept太慢 → 丢弃新连接

查看：

```
cat /proc/sys/net/ipv4/tcp_max_syn_backlog # 半连
ss -ltn # Recv-Q=当前全连接数, Send-Q=listen backlog
```

 **面试追问：**

1. HTTP和HTTPS的区别?
2. MQTT为什么适合IoT? QoS三个等级是什么?
3. CoAP和MQTT在嵌入式中怎么选?

嵌入式建议： 嵌入式网络协议选型：资源充足用MQTT(TCP,可靠),极度受限用CoAP(UDP,轻量)。HTTP仅用于配置页面/OTA下载。面试说清楚各自特点和选型依据。

Q11: 什么是SYN Flood攻击？如何防御？

 **秒懂：** SYN Flood用大量伪造SYN包填满半连接队列，正常连接无法建立。防御：SYN Cookie(不保存半连接状态，用加密Cookie验证)、增大队列、缩短超时、防火墙过滤。

 **面试高频** | 安全相关岗位必考 | 大疆/海康网络安全追问

SYN Flood是经典的DoS攻击方式：

攻击原理：

攻击者伪造大量SYN包(源IP伪造) → 服务器创建半连接 → 等待ACK
→ 半连接队列满 → 正常用户连接被拒绝

防御方法：

1. SYN Cookie：不保存半连接状态，通过时间戳+哈希验证ACK合法性

```
echo 1 > /proc/sys/net/ipv4/tcp_syncookies
```

2. 增大队列：

```
echo 4096 > /proc/sys/net/ipv4/tcp_max_syn_backlog
```

3. 减少SYN+ACK重试次数：

```
echo 1 > /proc/sys/net/ipv4/tcp_synack_retries
```

Q12: UDP如何实现可靠传输？

 **面试高频** | 牛客高频 | 游戏/音视频方向常考

 **秒懂：** UDP本身不可靠。应用层实现可靠传输：序列号+确认+超时重传+乱序缓存。参考QUIC/KCP协议。嵌入式中可用简单的确认重传机制：发一帧→等ACK→超时重发。

UDP本身不可靠，但应用层可以加可靠性(如QUIC, KCP)：

实现方案(仿TCP)：

1. 序列号：每个包编号
2. 确认应答：接收方发ACK
3. 超时重传：未收到ACK则重传
4. 滑动窗口：流量控制

简易可靠UDP伪代码：

发送方：

```
packet.seq = next_seq++;  
send(packet);  
timer_start(packet.seq, TIMEOUT);  
// 收到ACK → 确认，超时 → 重传
```

接收方：

```
recv(packet);  
if (packet.seq == expected_seq) {  
    deliver(packet.data);  
    expected_seq++;  
}
```

```
} else {  
    // 乱序：缓存或丢弃  
}  
send_ack(expected_seq);
```

Q13: OSI七层模型和TCP/IP四层模型?

💡 面试高频 | 网络基础第一题 | 必须能说出每层干什么

💡 秒懂：OSI七层(物理→数据链路→网络→传输→会话→表示→应用)是理论模型。TCP/IP四层(网络接口→网络→传输→应用)是实际使用。面试画图+对应关系+每层协议举例。

网络分层是面试基础题：

OSI七层：	TCP/IP四层：	协议示例：
应用层	应用层	HTTP/FTP/MQTT/DNS
表示层		SSL/TLS
会话层		RPC
传输层	传输层	TCP/UDP
网络层	网络层	IP/ICMP/ARP
数据链路层	网络接口层	Ethernet/WiFi/PPP
物理层		电缆/光纤

Q14: IP地址分类和子网划分?

💡 秒懂：IPv4地址32位分为网络号+主机号。子网掩码划分子网(如255.255.255.0表示前24位是网络号)。CIDR表示法：192.168.1.0/24。私有地址：10.x/172.16-31.x/192.168.x.x。

网络工程师面试必备(嵌入式IoT也常涉及)：

IPv4地址分类：

A类：1.0.0.0 ~ 126.255.255.255 /8 (大型网络)
B类：128.0.0.0 ~ 191.255.255.255 /16 (中型网络)
C类：192.0.0.0 ~ 223.255.255.255 /24 (小型网络)
D类：224~239 (多播)
E类：240~255 (保留)

私有地址：

10.0.0.0/8
172.16.0.0/12
192.168.0.0/16

子网划分示例：

192.168.1.0/24 分成4个子网：

- 192.168.1.0/26 (0~63, 可用1~62)
- 192.168.1.64/26 (64~127)
- 192.168.1.128/26 (128~191)
- 192.168.1.192/26 (192~255)

Q15: ARP协议的工作原理?

 **秒懂：** ARP将IP地址解析为MAC地址：广播'谁是192.168.1.1?' → 目标回复自己的MAC → 缓存到ARP表。同一局域网内通信必须知道对方MAC——ARP就是翻译官。

ARP(地址解析协议)将IP地址映射为MAC地址：

场景：主机A(192.168.1.1)想发数据给主机B(192.168.1.2)

1. A查ARP缓存：有B的MAC吗？
 - ↳ 有：直接用
 - ↳ 没有：发送ARP请求(广播)
2. ARP请求(广播)：
"谁是192.168.1.2? 请告诉192.168.1.1(MAC: AA:BB:CC:DD:EE:FF)"
3. B收到后回复(单播)：
"我是192.168.1.2, 我的MAC是11:22:33:44:55:66"
4. A更新ARP缓存, 后续直接用

命令：

```
arp -a          # 查看ARP表
arping 192.168.1.2 # 主动探测
```

Q16: ICMP协议的作用?

 **秒懂：** ICMP是IP层的'信差'——报告网络错误和状态。ping用ICMP Echo请求/回复测连通性，traceroute用ICMP TTL超时报文探测路由。网络诊断的基础协议。

ICMP(互联网控制消息协议)用于网络诊断和错误报告：

ICMP类型	作用	工具
Echo Request/Reply	连通性检测	ping
Destination Unreachable	主机/端口不可达	自动报告
Time Exceeded	TTL减为0	traceroute
Redirect	路由重定向	路由器发送

```
ping -c 3 192.168.1.1      # ICMP Echo
traceroute 8.8.8.8          # 利用TTL递增发送UDP/ICMP
```

Q17: NAT(网络地址转换)的原理?

 **秒懂：** NAT把私有IP转换为公网IP(一个公网IP供多台设备共享上网)。路由器维护转换表：内网IP:端口↔公网IP:端口。嵌入式设备通常在NAT内网中，外部主动连入需要端口映射。

NAT让私网主机通过少量公网IP访问互联网：

内网主机：	NAT设备：	外网服务器：
192.168.1.2:5000	→ 转换为202.1.1.1:10001	→ 8.8.8.8:53
192.168.1.3:6000	→ 转换为202.1.1.1:10002	→ 8.8.8.8:53

NAT表：

内网地址:端口	公网地址:端口
192.168.1.2:5000	↔ 202.1.1.1:10001
192.168.1.3:6000	↔ 202.1.1.1:10002

嵌入式相关：IoT设备在内网，需要NAT穿透才能被外部访问
穿透方法：STUN/TURN/ICE，或MQTT等长连接协议

Q18: DNS解析过程?

 **秒懂：** 浏览器输入域名→查本地DNS缓存→问本地DNS服务器→递归/迭代查询（根→顶级→权威DNS）→返回IP地址。嵌入式IoT设备连云服务器时也走DNS解析。

域名解析流程(面试可能问递归查询和迭代查询的区别)：

浏览器输入www.example.com：

1. 查浏览器DNS缓存
2. 查操作系统DNS缓存(/etc/hosts)
3. 查本地DNS服务器(递归解析器)
4. 本地DNS → 根服务器："谁管.com?"
5. 本地DNS → .com顶级域："谁管example.com?"
6. 本地DNS → example.com权威服务器："www.example.com的IP是?"
7. 返回IP地址,缓存TTL时间

嵌入式中：设备通常用静态DNS或mDNS(局域网设备发现)

Q19: HTTP协议基础?

 **秒懂：** HTTP是无状态的请求-响应协议(基于TCP)。请求：方法(GET/POST)+URL+头部+体。响应：状态码(200/404/500)+头部+体。嵌入式HTTP通常用轻量库(如microhttpd)。

嵌入式设备越来越多提供Web接口(RESTful API)：

HTTP请求格式：

```
GET /api/status HTTP/1.1\r\n
Host: 192.168.1.100\r\n
Content-Type: application/json\r\n
\r\n
```

HTTP响应格式：

```
HTTP/1.1 200 OK\r\n
```

```
Content-Type: application/json\r\n
Content-Length: 25\r\n
\r\n
{"status":"running"}
```

常用状态码：

200 OK
301 永久重定向
400 请求错误
403 禁止
404 未找到
500 服务器错误

Q20: HTTPS和TLS的工作原理？

 **秒懂：** HTTPS = HTTP + TLS加密。TLS握手：交换证书→验证身份→协商密钥→加密通信。嵌入式IoT设备连接云服务必须用TLS(如MQTT over TLS)，用mbedtls库实现。

嵌入式设备安全通信需要TLS：

TLS握手简化过程：

1. Client Hello：支持的加密套件列表，随机数
2. Server Hello：选定加密套件，证书，随机数
3. 客户端验证证书(CA签名)
4. 客户端生成预主密钥(用服务器公钥加密发送)
5. 双方用三个随机数生成会话密钥
6. 后续用对称加密通信(AES等，高效)

嵌入式：

- mbedtls/wolfSSL(轻量级TLS实现)
- 证书体积和内存是挑战
- 硬件加密加速(AES-NI/ARM CE)

二、网络编程进阶（Q21~Q40）

 **面试追问：**

1. ARP欺骗攻击原理？
2. 嵌入式设备如何防御ARP攻击？
3. RARP是什么,现在还用吗？

嵌入式建议： 工业嵌入式设备(PLC/网关)暴露在网络中容易被ARP攻击。防御方法：静态ARP表、ARP检测、VLAN隔离。面试可作为安全意识的加分项。

Q21: socket编程中常见的错误和处理？

 **秒懂：** 常见错误：ECONNREFUSED(对方没有监听)、ETIMEDOUT(连接超时)、EADDRINUSE(端口被占)、EPIPE(对方已关闭)、EAGAIN(非阻塞暂时无数据)。每个errno要有对应处理策略。

嵌入式网络编程中常遇到的errno：

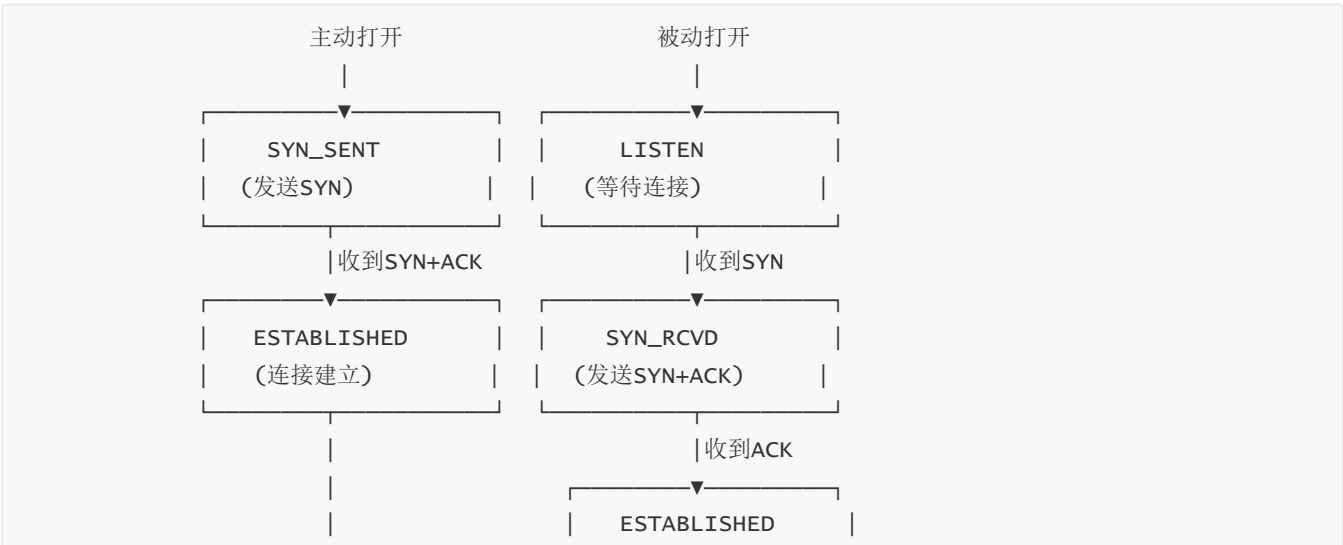
errno	含义	处理方法
ECONNREFUSED	对方拒绝连接	检查服务是否启动
ETIMEDOUT	连接超时	重试/检查网络
ECONNRESET	对方强制关闭	重连
EPIPE	写入已关闭的socket	捕获SIGPIPE
EAGAIN	非阻塞无数据	继续epoll等待
EINTR	被信号中断	重试

```
// 健壮的读取函数
ssize_t read_n(int fd, void *buf, size_t count) {
    size_t left = count;
    char *ptr = buf;
    while (left > 0) {
        ssize_t n = read(fd, ptr, left);
        if (n < 0) {
            if (errno == EINTR) continue;
            if (errno == EAGAIN) break;
            return -1;
        }
        if (n == 0) break; // 对端关闭
        left -= n;
        ptr += n;
    }
    return count - left;
}
```

Q22: TCP连接的状态转换图?

秒懂：CLOSED→SYN_SENT→ESTABLISHED→FIN_WAIT_1→FIN_WAIT_2→TIME_WAIT→CLOSED(主动关闭方)。CLOSED→LISTEN→SYN_RCVD→ESTABLISHED→CLOSE_WAIT→LAST_ACK→CLOSED(被动关闭方)。

面试必须能画出TCP状态机：



关闭连接：
主动关闭： ESTABLISHED→FIN_WAIT_1→FIN_WAIT_2→TIME_WAIT→CLOSED
被动关闭： ESTABLISHED→CLOSE_WAIT→LAST_ACK→CLOSED

全网最详细的嵌入式实习/秋招/春招公司清单，正在更新中，需要可关注微信公众号：嵌入式校招菌。

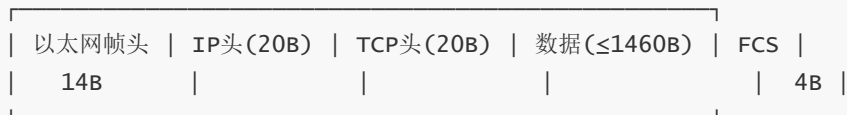
Q23: 什么是TCP的MSS和MTU?

 **秒懂：** MTU是链路层一帧能装的最大数据量(以太网1500字节)。MSS是TCP一个段能装的最大数据量(MTU-IP头-TCP头=1460字节)。超过MTU会IP分片，影响性能。

理解MTU/MSS对嵌入式网络调优很重要：

MTU(Maximum Transmission Unit)：数据链路层最大帧长度
以太网MTU = 1500字节

MSS(Maximum Segment Size)：TCP单次发送最大数据量
 $MSS = MTU - IP头(20) - TCP头(20) = 1460$ 字节



如果数据>MTU → IP分片(尽量避免,影响性能)
TCP通过MSS协商避免IP层分片

Q24: 什么是Socket的backlog参数?

 **秒懂：** backlog是listen()的参数，指定已完成握手待accept的连接队列长度。设太小会在高并发时拒绝连接。嵌入式服务器根据预期并发量设置(通常不大，5-128)。

listen(fd, backlog)中backlog的含义：

```
// backlog定义了全连接队列的最大长度
listen(listen_fd, 128); // 最多允许128个完成三次握手但未accept的连接

// 如果队列满：新的SYN可能被丢弃或返回RST
// 嵌入式设备：根据预期并发连接数设置合理值
// 高并发服务器：设置较大值(如1024)
```

Q25: 广播和多播的区别?

 **秒懂：** 广播发给子网内所有设备(一对所有)，多播发给特定组的设备(一对多)。多播更节省带宽——只有加入多播组的设备才收到数据。嵌入式设备发现常用多播。

嵌入式局域网通信中常用：

	广播(Broadcast)	多播(Multicast)
范围	同一子网所有主机	加入多播组的主机

	广播(Broadcast)	多播(Multicast)
地址	255.255.255.255或子网广播	224.0.0.0~239.255.255.255
路由	不跨路由器	可跨路由器(IGMP)
效率	低(所有主机都处理)	高(只有组员处理)
用途	设备发现、DHCP	视频流、传感器数据分发

Q26: TCP和UDP的头部格式？

 **秒懂：** TCP头20字节(源端口/目标端口/序号/确认号/标志位/窗口大小/校验和)。UDP头8字节(源端口/目标端口/长度/校验和)。UDP头部简单所以开销小。

理解报文格式对wireshark抓包分析很重要：

TCP头部(20字节+可选)：

源端口(16)	目的端口(16)	
序列号(32)		
确认号(32)		
头部长度 保留 URG ACK PSH RST SYN FIN 窗口(16)		
校验和(16)	紧急指针(16)	

UDP头部(8字节)：

源端口(16)	目的端口(16)	
长度(16)	校验和(16)	

Q27: 什么是DHCP？ 工作流程？

 **秒懂：** DHCP自动分配IP地址：DISCOVER(客户端广播找DHCP服务器)→OFFER(服务器提供IP)→REQUEST(客户端选择并请求)→ACK(服务器确认)。嵌入式联网设备通常用DHCP自动获取IP。

DHCP自动分配IP地址(嵌入式设备入网常用)：

DHCP四步交互(DORA):

1. Discover(广播): 客户端:"谁是DHCP服务器?"
2. Offer(广播): 服务器:"我可以给你192.168.1.100"
3. Request(广播): 客户端:"我要192.168.1.100"
4. ACK(广播): 服务器:"确认,有效期24小时"

嵌入式中:

- 设备启动时通过DHCP获取IP
- 服务器可根据MAC分配固定IP(静态绑定)
- 没有DHCP服务器: 使用链路本地地址(169.254.x.x)

Q28: Wireshark/tcpdump抓包分析?

 **秒懂:** tcpdump命令行抓包: tcpdump -i eth0 port 80 -w file.pcap。Wireshark图形化分析pcap文件——过滤条件+协议解析+流追踪。嵌入式网络调试的必备技能。

嵌入式网络问题排查的核心技能:

```
# tcpdump抓包(嵌入式开发板)
tcpdump -i eth0 -w capture.pcap           # 抓所有包
tcpdump -i eth0 port 8080                  # 指定端口
tcpdump -i eth0 host 192.168.1.100         # 指定主机
tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0' # 只看SYN包

# Wireshark过滤示例
tcp.port == 8080
ip.addr == 192.168.1.100
tcp.flags.syn == 1 and tcp.flags.ack == 0   # 新连接SYN
tcp.analysis.retransmission                 # 重传包
```

Q29: 什么是WebSocket? 和HTTP的关系?

 **秒懂:** WebSocket在HTTP基础上升级为全双工通信——服务器可以主动推送数据(HTTP只能客户端请求)。嵌入式Web控制界面常用WebSocket实时推送传感器数据。

WebSocket是全双工通信协议(嵌入式Web实时数据推送):

HTTP: 请求-响应模型(客户端主动)

WebSocket: 双向实时通信(服务器可主动推送)

建立过程(HTTP升级):

```
客户端: GET /ws HTTP/1.1
        Upgrade: websocket
        Connection: Upgrade
服务器: HTTP/1.1 101 Switching Protocols
        Upgrade: websocket
```

嵌入式应用:

- 实时传感器数据推送到web页面
- 远程控制命令下发
- 比轮询HTTP效率高得多

Q30: 什么是MQTT协议？嵌入式IoT中的应用？

 **秒懂：** MQTT是轻量级发布/订阅消息协议：设备publish发布消息→Broker中转→订阅者subscribe接收。最小报文2字节。嵌入式IoT首选协议，适合低带宽、不稳定网络。

MQTT是轻量级发布/订阅消息协议，IoT设备最常用：

架构：

设备(Publisher) → Broker(服务器) → 订阅者(Subscriber)

特点：

- 小开销(最小2字节头)
- 发布/订阅模式(解耦)
- QoS三级(0:最多一次, 1:至少一次, 2:恰好一次)
- 支持遗嘱消息(will, 设备异常断开时通知)
- 保持连接(Keep Alive心跳)

嵌入式典型用法：

传感器 → 发布 sensor/temp → Broker → 订阅者(云平台/APP)

APP → 发布 device/cmd → Broker → 订阅者(设备)

Q31: lwIP协议栈在嵌入式中的使用？

 **秒懂：** lwIP是轻量级TCP/IP协议栈(几十KB RAM)，适合没有OS或RTOS的嵌入式设备。支持TCP/UDP/DHCP/DNS等核心协议。STM32+lwIP是嵌入式网络的经典组合。

lwIP是MCU级嵌入式设备的轻量TCP/IP协议栈：

```
// lwIP基本使用(raw API)
#include "lwip/tcp.h"

struct tcp_pcb *pcb = tcp_new();
tcp_bind(pcb, IP_ADDR_ANY, 8080);
pcb = tcp_listen(pcb);
tcp_accept(pcb, accept_callback);

err_t accept_callback(void *arg, struct tcp_pcb *newpcb, err_t err) {
    tcp_recv(newpcb, recv_callback);
    return ERR_OK;
}

err_t recv_callback(void *arg, struct tcp_pcb *pcb, struct pbuf *p, err_t err) {
    if (p != NULL) {
        // 处理数据
        tcp_write(pcb, response, resp_len, TCP_WRITE_FLAG_COPY);
        pbuf_free(p);
    }
    return ERR_OK;
}
```

lwIP vs Linux TCP/IP：

- lwIP适合MCU(RAM<100KB), 裸机或RTOS
- Linux TCP/IP功能完整, 适合Cortex-A等高性能平台

Q32: CoAP协议(嵌入式RESTful)?

 **秒懂:** CoAP是面向受限设备的轻量级RESTful协议(基于UDP)。类似HTTP但报文极小(4字节头)。支持GET/PUT/POST/DELETE、观察模式(observe)。适合传感器数据上报。

CoAP是为受限设备设计的轻量级RESTful协议:

CoAP vs HTTP:

- 运行在UDP上(比TCP轻量)
- 二进制格式(HTTP是文本)
- 4字节头(HTTP头可能几百字节)
- 支持资源观察(类似订阅)

CoAP方法: GET/POST/PUT/DELETE (和HTTP相同)

嵌入式场景:

```
GET coap://sensor.local/temperature
→ 2.05 Content "23.5"
```

Q33: TCP连接的优雅关闭和强制关闭?

 **秒懂:** 优雅关闭: shutdown(SHUT_WR)告诉对方'我发完了但还能收'→等对方也关闭→close。强制关闭: SO_LINGER设置超时0后close, 直接发RST。嵌入式中优雅关闭避免数据丢失。

关闭策略影响数据完整性:

```
// 优雅关闭(等待数据发完)
shutdown(fd, SHUT_WR); // 关闭写端,发FIN
// 继续读取对方剩余数据...
while (read(fd, buf, sizeof(buf)) > 0);
close(fd);

// 强制关闭(RST, 丢弃未发数据)
struct linger lg = { .l_onoff = 1, .l_linger = 0 };
setsockopt(fd, SOL_SOCKET, SO_LINGER, &lg, sizeof(lg));
close(fd); // 发RST而非FIN

// 半关闭
shutdown(fd, SHUT_WR); // 关写不关读(常用于通知对方数据发完)
shutdown(fd, SHUT_RD); // 关读不关写
```

Q34: 网络编程中的大端小端问题?

 **秒懂:** 网络字节序是大端, x86/ARM小端。发送前htonl/htons转网络序, 接收后ntohl/ntohs转主机序。多字节字段(IP/端口/长度)都需要转换, 单字节不需要。

嵌入式跨设备通信经常遇到字节序问题:

```
// 网络字节序=大端
// x86/ARM(通常)=小端

// 不同字节序的表示：
// 数值：0x12345678
// 大端(网络)：[12][34][56][78]（高位在前）
// 小端(主机)：[78][56][34][12]（低位在前）

// 转换
uint32_t host_val = 0x12345678;
uint32_t net_val = htonl(host_val); // 主机→网络

// 结构体网络传输(必须处理字节序)
typedef struct __attribute__((packed)) {
    uint16_t cmd; // 必须htons
    uint32_t param; // 必须htonl
} NetPacket;
```

Q35: 什么是零窗口和窗口探测？

 **秒懂：** 零窗口：接收方缓冲区满时通告窗口=0，发送方停止发送。窗口探测：发送方定期发1字节探测包检查窗口是否恢复。防止因窗口更新报文丢失导致的死锁。

TCP流控的边界情况：

零窗口场景：

接收方处理慢 → 接收缓冲满 → 通告窗口=0 → 发送方停止

发送方如何知道窗口恢复？

→ 窗口探测机制(Persist Timer)：

定期发送1字节探测包 → 等待对方回复新窗口值

问题：如果窗口更新ACK丢失 → 死锁(双方互相等待)

→ 所以需要Persist Timer

Q36: 什么是带外数据(OOB/紧急数据)？

 **秒懂：** 带外数据(TCP紧急指针)标记'紧急'数据——接收方优先处理。实际很少使用(只能传1字节且实现不一致)。嵌入式中不建议使用，用自定义协议的优先级字段替代。

TCP的紧急数据机制(实际使用较少)：

```
// 发送紧急数据
send(fd, "X", 1, MSG_OOB);

// 接收紧急数据
recv(fd, buf, 1, MSG_OOB);

// 或通过SIGURG信号通知
signal(SIGURG, oob_handler);
fcntl(fd, F_SETOWN, getpid());

// 实际应用：Telnet中Ctrl+C中断、FTP中止传输
// 嵌入式：紧急停止命令
```

Q37: 什么是端口复用和SO_REUSEPORT?

 **秒懂：** SO_REUSEADDR允许绑定处于TIME_WAIT的端口(服务器重启不用等)。SO_REUSEPORT允许多个进程/线程绑定同一端口(负载均衡)。嵌入式服务器必须设SO_REUSEADDR。

端口复用在高并发中的作用：

```
// SO_REUSEADDR：TIME_WAIT状态也能绑定(基本必设)
int opt = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// SO_REUSEPORT：多个socket绑定同一端口(内核负载均衡)
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));
// 多进程/多线程各自listen同一端口 → 内核分发连接
```

Q38: IPv6基础知识?

 **秒懂：** IPv6地址128位(解决IPv4地址不够的问题)。简化的头部、内置安全性(IPsec)、支持自动配置(SLAAC)。嵌入式IoT的大规模部署未来必然用IPv6(如Thread协议)。

IoT设备越来越多使用IPv6：

IPv6地址：128位，如 2001:0db8:85a3::8a2e:0370:7334
IPv4地址：32位，如 192.168.1.1

IPv6优势：

- 地址空间巨大(每个IoT设备都可有全球唯一地址)
- 无需NAT(端到端直达)
- 内置IPsec
- 自动配置(SLAAC)

嵌入式：

```
struct sockaddr_in6 addr6;
addr6.sin6_family = AF_INET6;
inet_pton(AF_INET6, ":::1", &addr6.sin6_addr);
```

Q39: 什么是心跳包设计?

 **秒懂：**心跳包是应用层定期发送的'我还活着'消息。目的：①检测连接是否正常 ②防止NAT/防火墙超时断开。设计要点：间隔适当(10-30秒)、超时几次判断断开、重连机制。

嵌入式设备长连接必须实现心跳机制：

```
// 应用层心跳设计
#define HEARTBEAT_INTERVAL 30 // 30秒
#define HEARTBEAT_TIMEOUT 90 // 3倍间隔无心跳则断开

// 发送心跳
void *heartbeat_thread(void *arg) {
    while (connected) {
        send_heartbeat_packet(fd);
        sleep(HEARTBEAT_INTERVAL);
    }
}

// 检测心跳超时
void check_timeout(void) {
    time_t now = time(NULL);
    if (now - last_heartbeat_time > HEARTBEAT_TIMEOUT) {
        // 连接断开,执行重连
        reconnect();
    }
}
```

Q40: 什么是Protobuf/MessagePack? 嵌入式中的应用?

 **秒懂：**Protobuf(Google)和MessagePack是高效二进制序列化格式——比JSON体积小很多、解析快。嵌入式中用于低带宽网络传输(如IoT数据上报)和存储紧凑的结构化数据。

高效的序列化协议(替代JSON)：

JSON:	{ "temp":23.5, "humi":65 }	→	28字节(文本)
Protobuf:	[二进制编码]	→	~8字节(紧凑)
MessagePack:	[二进制]	→	~12字节

嵌入式优势：

- 更小体积(节省带宽)
- 更快解析(无字符串解析)
- 跨语言兼容(设备C/云端Python)

嵌入式常用：

- nanopb(C语言Protobuf实现, 适合MCU)
- CBOR(类JSON的二进制格式)
- FlatBuffers(零拷贝反序列化)

★ 嵌入式/IoT协议选型对比 (系统设计题常考)：

协议	层级	适用场景	特点	典型设备
MQTT	应用层	IoT云端通信	轻量/发布订阅/QoS	智能家居/传感器
CoAP	应用层	资源受限IoT	类REST/UDP/低功耗	NB-IoT设备
HTTP	应用层	Web设备管理	通用/重量级	路由器/网关
Modbus	应用层	工业控制	简单/主从式	PLC/变频器
WebSocket	应用层	实时推送	全双工/持久连接	实时监控面板

三、嵌入式网络协议（Q41~Q55）

Q41: MQTT QoS三级的区别和实现？

 **秒懂：** QoS 0(最多一次，可能丢)→QoS 1(至少一次，可能重复)→QoS 2(恰好一次，最可靠但最慢)。嵌入式中传感器数据用QoS 0/1即可，控制命令用QoS 1或2确保送达。

 **面试高频** | IoT/智能家居岗位必考 | 涂鸦/小米IoT面经常见

MQTT三种服务质量等级的对比：

QoS	含义	实现	适用场景
0	最多一次 (Fire&Forget)	直接发,不确认	传感器频繁上报(丢一条无所谓)
1	至少一次	等PUBACK,否则重发	控制命令(可能重复)
2	恰好一次	四次握手 (PUBREC/PUBREL/PUBCOMP)	计费/支付(不能重复)

QoS 1流程：
Publisher → PUBLISH(msgid=1) → Broker
Publisher ← PUBACK(msgid=1) ← Broker
(如果超时没收到PUBACK → 重发PUBLISH)

QoS 2流程(四步)：
Publisher → PUBLISH → Broker
Publisher ← PUBREC ← Broker (收到了)
Publisher → PUBREL → Broker (确认释放)
Publisher ← PUBCOMP ← Broker (完成)

Q42: MQTT的遗嘱(Will)消息？

 **秒懂：** 遗嘱消息：设备连接Broker时预设'我掉线后发这条消息'。设备异常断开时Broker自动发布遗嘱消息通知订阅者。用于设备离线检测和告警。

遗嘱消息用于设备异常离线时通知其他订阅者：

连接时设置遗嘱：

```
CONNECT {
  will_topic: "device/status"
  will_message: "offline"
  will_qos: 1
  will_retain: true
}
```

正常断开：发送DISCONNECT → Broker不发遗嘱

异常断开：TCP断开/超时 → Broker自动发布遗嘱消息

应用场景：

- IoT设备状态监控(在线/离线)
- 设备故障告警

Q43: HTTP RESTful API设计(嵌入式Web)?

 **秒懂**：RESTful API：用HTTP方法操作资源。GET /api/sensor(读)、PUT /api/led(写)、POST(创建)、DELETE(删除)。嵌入式Web服务器提供JSON格式的API供前端/APP调用。

嵌入式设备提供Web管理接口的设计：

RESTful规范：

GET	/api/sensors	→ 获取所有传感器数据
GET	/api/sensors/1	→ 获取传感器1数据
POST	/api/config	→ 更新配置
PUT	/api/device/reboot	→ 重启设备
DELETE	/api/logs	→ 清除日志

响应格式：

```
{
  "code": 200,
  "data": {
    "temperature": 23.5,
    "humidity": 65,
    "timestamp": 1703001234
  }
}
```

嵌入式实现：

- 轻量级HTTP库：mongoose, libmicrohttpd
- 嵌入式Web框架：Boa, uhttpd(OpenWrt)

Q44: Modbus协议(工业嵌入式)?

 **秒懂**：Modbus是工业控制领域的标准通信协议。RTU(串口二进制)和TCP(以太网)两种模式。功能码：03读寄存器、06写单个寄存器、16写多个寄存器。PLC和传感器的通用语言。

 **面试高频** | 工业自动化/PLC岗位必考 | 汇川/中控面经

Modbus是工业自动化最常用的通信协议：

Modbus RTU帧格式(RS-485/串口):

[地址(1B)] [功能码(1B)] [数据(nB)] [CRC(2B)]

常用功能码:

0x01: 读线圈(开关量)

0x03: 读保持寄存器(模拟量)

0x06: 写单个寄存器

0x10: 写多个寄存器

示例(读保持寄存器):

请求: 01 03 00 00 00 02 C4 0B

地址 功能 起始 数量 CRC

响应: 01 03 04 00 64 00 FF xx xx

地址 功能 字节数 数据 CRC

Modbus TCP: 在TCP上传输, 去掉CRC加MBAP头

Q45: mDNS/DNS-SD(局域网设备发现)?

 **秒懂:** mDNS(多播DNS)让局域网设备用名字互相发现(如device.local)。DNS-SD(服务发现)发现设备提供的服务(如打印机、摄像头)。嵌入式设备免配IP直接用名字访问。

无需配置DNS服务器的局域网设备互相发现:

mDNS:

设备宣告: "我是mydevice.local, IP是192.168.1.50"

其他设备解析: ping mydevice.local → 192.168.1.50

使用多播地址: 224.0.0.251:5353

DNS-SD(Service Discovery):

设备宣告服务: "_http._tcp.local" → "MyDevice web Server"

其他设备查询: "有哪些HTTP服务?" → 列出所有

嵌入式应用:

- 打印机发现(AirPrint)
- IoT设备自动发现(不需要知道IP)
- Apple Bonjour / Linux Avahi

Q46: TLS/DTLS在嵌入式中的实现?

 **秒懂:** 嵌入式TLS用mbedTLS或wolfSSL(体积小适合MCU)。DTLS是基于UDP的TLS(保持数据报语义)。证书验证可能耗时(几秒)——预加载根证书和会话缓存优化。

嵌入式设备的安全通信:

mbedTLS(轻量级, 适合MCU):

- 支持TLS 1.2/1.3
- 可裁剪(只编译需要的算法)
- RAM使用可控(~50KB起)

wolfSSL:

- 更小的代码体积

- 支持FIPS认证
- 适合安全要求高的场景

DTLS(TLS over UDP):

- CoAP/MQTT-SN的安全层
- 适合丢包环境(重传机制)

证书管理:

- 设备出厂预置CA根证书
- 设备私钥存在安全芯片(SE)中
- 在线证书验证(OCSP)

Q47: 网络协议自定义设计原则?

 **秒懂:** 自定义协议设计原则: 帧头(同步字)→版本→类型→长度→数据→校验(CRC/Checksum)。可扩展(预留字段)、可解析(长度明确)、可校验(防止传输错误)。

嵌入式产品经常需要设计私有协议:

协议设计要素:

1. 帧头/帧尾(定界): 0xAA55...55AA
2. 长度字段(分包)
3. 命令字段(区分功能)
4. 序列号(重传/去重)
5. 校验(CRC/校验和)
6. 版本号(兼容升级)

典型帧格式:

帧头	版本	长度	序号	命令	数据	CRC
2B	1B	2B	2B	1B	nB	2B

状态机解析(适合串口/网络字节流):

WAIT_HEAD → RECV_LEN → RECV_DATA → CHECK_CRC → DISPATCH

Q48: WebSocket在嵌入式设备中的实现?

 **秒懂:** 嵌入式WebSocket用轻量库(如libwebsockets)。流程: HTTP升级握手→全双工帧收发。适合嵌入式Web配置界面的实时数据推送(比轮询HTTP高效得多)。

嵌入式设备提供实时Web数据推送:

```
// 嵌入式WebSocket简化实现(基于mongoose库)
#include "mongoose.h"

void event_handler(struct mg_connection *c, int ev, void *ev_data) {
    if (ev == MG_EV_HTTP_MSG) {
        struct mg_http_message *hm = ev_data;
        if (mg_http_match_uri(hm, "/ws")) {
            mg_ws_upgrade(c, hm, NULL);
        }
    }
}
```

```

    } else if (ev == MG_EV_WS_MSG) {
        struct mg_ws_message *wm = ev_data;
        // 收到WebSocket消息
        mg_ws_send(c, "ACK", 3, WEBSOCKET_OP_TEXT);
    }
}

```

Q49: SNMP网络管理协议?

 **秒懂：** SNMP(简单网络管理协议)用于监控和管理网络设备。Agent运行在设备上→Manager查询/设置MIB对象→Trap主动告警。大型网络设备和工业设备常用。

嵌入式网络设备管理(路由器/交换机):

SNMP组成:

- Agent(设备端): 响应查询/发送Trap
- Manager(管理站): 查询/设置/接收告警
- MIB(管理信息库): 设备可管理的参数树

操作:

GET: 读取参数值
SET: 修改参数
TRAP: 设备主动上报异常

OID示例:

.1.3.6.1.2.1.1.1.0 → 系统描述
.1.3.6.1.2.1.2.2.1.10.1 → 接口1的入包数

Q50: gRPC在嵌入式中的应用?

 **秒懂：** gRPC基于HTTP/2+Protobuf, 高效的RPC框架。嵌入式中高性能设备(如边缘计算)可以用gRPC与云端通信。资源受限MCU更适合用MQTT或CoAP。

高性能RPC框架(适合嵌入式网关/边缘设备):

```

// 定义服务(sensor.proto)
syntax = "proto3";
service SensorService {
    rpc GetData(SensorRequest) returns (SensorData);
    rpc StreamData(SensorRequest) returns (stream SensorData);
}
message SensorRequest { int32 sensor_id = 1; }
message SensorData { float temperature = 1; float humidity = 2; }

```

优势:

- Protobuf序列化(高效紧凑)
- HTTP/2(多路复用/流式)
- 跨语言(设备C/云端Go/Python)
- 适合微服务架构的边缘网关

Q51: 蓝牙BLE协议栈?

 **秒懂：** BLE(低功耗蓝牙)协议栈：物理层→链路层→HCI→L2CAP→ATT/GATT→应用。GATT定义Service和Characteristic的层次结构。BLE是可穿戴设备和传感器的首选短距通信。

BLE(低功耗蓝牙)是嵌入式近场通信主流：

BLE协议栈：

应用层：GATT Profile(通用属性配置文件)

GAP：广播/连接管理

ATT：属性传输

L2CAP：逻辑链路

HCI：主控接口

链路层：广告/数据信道

物理层：2.4GHz ISM频段

GATT模型：

Service(服务) → Characteristic(特征值) → Descriptor(描述)

示例(温度传感器)：

Service：温度服务(UUID：0x1809)

├─ Characteristic：当前温度(Read/Notify)

| └─ value：23.5°C

└─ Characteristic：采集间隔(Read/Write)

└─ value：1000ms

Q52: WiFi连接和配网方案?

 **秒懂：** WiFi连接：扫描→认证→关联→获取IP(DHCP)。配网方案：SmartConfig(手机广播)、AP模式(设备开热点手机连入配置)、BLE配网。嵌入式WiFi模块如ESP32常用SmartConfig。

嵌入式WiFi设备的配网是产品化难点：

配网方案：

1. AP模式：设备开热点→手机连接→配置WiFi信息→切回
2. SmartConfig：手机APP广播WiFi密码→设备嗅探获取
3. BLE辅助：通过BLE传递WiFi凭证(更可靠)
4. WPS：按路由器WPS键→自动配对(简单但不安全)
5. 二维码：扫码获取设备信息→APP配网

代码流程(Linux wpa_supplicant)：

```
// 生成配置
echo 'network={ssid="MyWiFi" psk="password"}' > /etc/wpa_supplicant.conf
// 启动连接
wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf
dhclient wlan0 // 获取IP
```

Q53: 网络拥塞控制算法对比？

秒懂： 拥塞控制算法演进：TCP Reno(经典)→TCP CUBIC(Linux默认，适合高带宽)→BBR(Google，基于带宽估计，高延迟网络表现好)。嵌入式IoT网络通常带宽低用默认CUBIC即可。

不同场景选择不同拥塞控制算法：

算法	特点	适用
Reno	传统AIMD	一般场景
Cubic	Linux默认,快速恢复	大带宽高延迟
BBR	基于带宽估计	长肥管道/丢包网络
Vegas	基于延迟	低延迟场景

```
# Linux查看/设置拥塞算法
cat /proc/sys/net/ipv4/tcp_congestion_control
echo bbr > /proc/sys/net/ipv4/tcp_congestion_control
```

Q54: 嵌入式网络安全(常见攻击和防御)？

秒懂： 常见攻击：中间人攻击(→TLS加密)、重放攻击(→时间戳/序列号)、固件逆向(→加密固件)、缓冲区溢出(→输入验证)。嵌入式安全：最小攻击面+加密通信+安全启动。

嵌入式设备面临的网络安全威胁：

攻击	描述	防御
中间人(MITM)	窃听/篡改通信	TLS/证书验证
重放攻击	重发旧数据	时间戳/序列号/Nonce
缓冲区溢出	覆盖返回地址	输入校验/ASLR/栈保护
固件逆向	提取加密密钥	安全启动/加密存储
DoS	耗尽设备资源	速率限制/连接数限制

Q55: 网络性能指标和测试方法？

秒懂： 性能指标：带宽(最大传输速率)、延迟(RTT往返时间)、丢包率、抖动(延迟变化)。测试工具：iperf(带宽)、ping(延迟/丢包)、netperf(吞吐量)。嵌入式用iperf测网络性能。

嵌入式网络性能评估：

```
# 带宽测试
iperf3 -s # 服务端
iperf3 -c 192.168.1.1 -t 10 # 客户端测试10秒

# 延迟测试
ping -c 100 192.168.1.1 | tail -1
```

```
# rtt min/avg/max/mdev = 0.5/1.2/3.4/0.5 ms

# 丢包测试
ping -c 1000 -i 0.01 192.168.1.1 | grep "packet loss"

# 网络吞吐量计算
# 理论最大: bandwidth * (1 - packet_loss) / (RTT)
# 实际受制于: 窗口大小, 拥塞, CPU处理能力
```

四、网络安全与加密 (Q56~Q70)

Q56: 对称加密和非对称加密的区别？

 **秒懂：** 对称加密：加密和解密用同一把钥匙(AES/DES)，速度快。非对称加密：公钥加密私钥解密(RSA/ECC)，安全但慢。实际TLS先用非对称交换密钥，再用对称加密传输数据。

密码学基础(嵌入式安全通信必备)：

	对称加密	非对称加密
密钥	加密解密同一把	公钥加密,私钥解密
速度	快(硬件加速)	慢(数学运算复杂)
典型算法	AES, DES, ChaCha20	RSA, ECC, Ed25519
应用	数据加密	密钥交换/签名
密钥分发	困难(需安全通道)	公钥可公开

实际使用(混合)： TLS用非对称加密协商出对称密钥，后续通信用对称加密(兼顾安全性和性能)。

Q57: 哈希函数的作用？ 常见算法？

 **秒懂：** 哈希函数将任意长度数据映射为固定长度摘要(不可逆)。用途：数据完整性校验、密码存储、数字签名。常见算法：MD5(已不安全)、SHA-256(推荐)、CRC(校验用，非安全)。

哈希用于数据完整性校验：

特性：

- 固定长度输出

- 不可逆(不能从哈希值还原数据)

- 雪崩效应(输入微小变化→输出完全不同)

- 抗碰撞(不同输入很难产生相同哈希)

常用：

MD5：128位(已不安全,仅用于校验)

SHA-256：256位(安全标准)

CRC32：32位(错误检测,非加密)

嵌入式应用：

- 固件校验：SHA-256(firmware.bin) = 已知哈希？

- 密码存储: `bcrypt(password + salt)`
- 数据去重: 内容hash作为唯一标识

Q58: 数字签名的原理?

 **秒懂:** 数字签名: 发送方用私钥对数据哈希加密→接收方用公钥解密并对比哈希。验证了数据来源(身份认证)和完整性(未被篡改)。固件签名验证是安全启动的基础。

数字签名验证数据来源和完整性(嵌入式安全启动):

签名过程(发送方):

1. 计算数据的哈希: $H = \text{SHA256}(\text{data})$
2. 用私钥加密哈希: $\text{Sig} = \text{RSA_Encrypt}(H, \text{private_key})$
3. 发送: $\text{data} + \text{Sig}$

验证过程(接收方):

1. 用公钥解密签名: $H1 = \text{RSA_Decrypt}(\text{Sig}, \text{public_key})$
2. 计算数据哈希: $H2 = \text{SHA256}(\text{data})$
3. 比较: $H1 == H2?$ → 验证通过

嵌入式安全启动:

BootROM中固化公钥 → 验证U-Boot签名 → 验证kernel签名

Q59: X.509证书和PKI体系?

 **秒懂:** X.509证书包含公钥+身份信息+CA签名。PKI(公钥基础设施)是证书签发和验证的体系: 根CA→中间CA→终端证书。TLS握手时服务器发送证书链供客户端验证。

证书是公钥的可信包装(TLS/HTTPS基础):

证书内容:

- 持有者信息(设备ID/域名)
- 公钥
- 有效期
- CA签名(证书链上级签发)

证书链:

Root CA(自签,预置在设备中)
↳ 中间CA(Root签发)
↳ 设备证书(中间CA签发)

嵌入式设备证书管理:

- 出厂预置: 根CA证书 + 设备私钥/证书
- 运行时: TLS握手出示证书,验证对方证书
- 存储: 私钥存在安全元件(SE)/eFuse中

Q60: CRC校验的原理和实现?

 **秒懂：** CRC(循环冗余校验)用多项式除法计算校验值。发送方计算CRC附在数据后→接收方重新计算CRC对比。嵌入式中串口、SPI、CAN等协议都用CRC校验数据完整性。

CRC是嵌入式通信中最常用的错误检测码：

```
// CRC-16/MODBUS实现
uint16_t crc16_modbus(uint8_t *data, uint16_t len) {
    uint16_t crc = 0xFFFF;
    for (uint16_t i = 0; i < len; i++) {
        crc ^= data[i];
        for (uint8_t j = 0; j < 8; j++) {
            if (crc & 0x0001)
                crc = (crc >> 1) ^ 0xA001;
            else
                crc >>= 1;
        }
    }
    return crc;
}

// 使用(通信帧校验)
uint8_t frame[] = {0x01, 0x03, 0x00, 0x00, 0x00, 0x02};
uint16_t crc = crc16_modbus(frame, 6);
frame[6] = crc & 0xFF;           // CRC低字节
frame[7] = (crc >> 8) & 0xFF;    // CRC高字节
```

Q61: AES加密在嵌入式中的使用?

 **秒懂：** AES是最广泛使用的对称加密算法(128/192/256位密钥)。嵌入式中用硬件AES加速器(STM32有CRYPT模块)或软件库(mbedtls)。ECB模式不安全，用CBC或GCM模式。

AES是嵌入式最常用的对称加密算法：

```
// 使用mbedtls的AES-128-CBC
#include "mbedtls/aes.h"

uint8_t key[16] = "0123456789abcdef";
uint8_t iv[16] = "fedcba9876543210";
uint8_t plaintext[32] = "Hello, Embedded!";
uint8_t ciphertext[32];

mbedtls_aes_context ctx;
mbedtls_aes_init(&ctx);
mbedtls_aes_setkey_enc(&ctx, key, 128);
mbedtls_aes_crypt_cbc(&ctx, MBEDTLS_AES_ENCRYPT, 32, iv, plaintext, ciphertext);
mbedtls_aes_free(&ctx);
```

AES模式选择：

- ECB：简单但不安全(相同明文→相同密文)

- CBC: 安全, 需要IV(嵌入式常用)
- GCM: 加密+认证(AEAD,最推荐)

Q62: 嵌入式安全启动(Secure Boot)?

 **秒懂:** 安全启动: 每级引导程序验证下一级的签名。BootROM(ROM内固化)→验证U-Boot签名→U-Boot验证内核签名→内核验证rootfs。链式信任确保固件未被篡改。

安全启动链保证固件完整性:

安全启动链:

BootROM(不可修改) → 验证SPL签名 → SPL验证U-Boot → U-Boot验证Kernel

实现方法:

1. 密钥对: 服务器持有私钥(签名), 设备持有公钥(验证)
2. 签名: 每次构建固件时用私钥签名
3. 验证: 启动时每级验证下一级签名
4. 回退: 验证失败→不启动/回退旧版本

密钥安全存储:

- OTP/eFuse: 一次性可编程(写入后不可修改)
- Secure Element(SE): 硬件安全芯片
- TrustZone: ARM安全世界隔离

Q63: 嵌入式设备的密钥管理?

 **秒懂:** 嵌入式密钥管理挑战: 不能硬编码(反编译会泄露)。方案: 硬件安全模块(HSM/SE)、eFuse一次性编程、TrustZone安全存储。密钥绝不能出现在代码或日志中。

密钥安全是整个安全体系的基础:

密钥类型:

- 设备密钥对(身份认证)
- 通信会话密钥(每次连接不同)
- 固件签名密钥(服务器端保管)
- 存储加密密钥(保护本地数据)

存储方案:

最安全: 安全芯片(SE/TPM)→私钥永远不出芯片
次安全: ARM TrustZone隔离
基本: 加密存储在Flash中(用设备唯一ID派生密钥加密)

生成:

- 出厂注入(生产线安全环境)
- 设备自生成(TRNG/硬件随机数)

轮换:

- 定期更新证书/密钥
- 支持远程密钥更新(需认证通道)

Q64: 什么是HMAC? 和普通哈希的区别?

 **秒懂:** HMAC = 哈希(密钥 + 消息)。比普通哈希多了一个共享密钥——只有知道密钥的人才能生成和验证。用于API认证、消息完整性校验。常用HMAC-SHA256。

HMAC(Hash-based Message Authentication Code)提供消息认证:

普通哈希: $\text{SHA256}(\text{message}) \rightarrow$ 只能检测篡改

HMAC: $\text{SHA256}(\text{key} + \text{message}) \rightarrow$ 既检测篡改又验证发送者

公式: $\text{HMAC}(K, M) = H((K \oplus \text{opad}) || H((K \oplus \text{ipad}) || M))$

嵌入式应用:

- API请求签名(防篡改)
- OTA包校验(防伪造)
- 通信认证(双方共享密钥)

```
// mbedTLS HMAC-SHA256
#include "mbedtls/md.h"

uint8_t key[] = "secret_key";
uint8_t msg[] = "important data";
uint8_t hmac[32];

mbedtls_md_hmac(mbedtls_md_info_from_type(MBEDTLS_MD_SHA256),
                key, sizeof(key)-1, msg, sizeof(msg)-1, hmac);
```

Q65: 嵌入式设备固件加密?

 **秒懂:** 固件加密保护知识产权和安全: 用AES加密固件镜像→设备启动时用内置密钥解密→执行。密钥存储在安全硬件中(eFuse/TrustZone), 防止固件被逆向分析。

保护固件不被逆向和盗用:

固件保护方案:

1. 加密存储: AES加密固件存入Flash
→ 启动时BootROM用硬件密钥解密到RAM执行
2. 代码混淆: 增加逆向难度(非加密)
3. 读保护: MCU的读保护位(RDP/Code Protection)
→ 防止通过JTAG/SWD读出Flash内容
4. 固件签名: 验证完整性(不是加密内容)

组合方案:

签名(完整性) + 加密(保密性) + 读保护(物理防护)

Q66: 随机数生成(TRNG vs PRNG)?

秒懂： TRNG(真随机数)基于物理噪声(热噪声等)，安全性高。PRNG(伪随机数)基于算法(种子确定则输出确定)，速度快但可预测。加密场景必须用TRNG或安全PRNG(如HMAC-DRBG)。

加密安全要求高质量随机数：

PRNG(伪随机数, 软件)：

- 确定性算法(同种子→同序列)
- 快速，但可预测
- 用途：模拟/测试，非安全场景

TRNG(真随机数, 硬件)：

- 物理噪声源(热噪声/抖动)
- 不可预测
- 用途：密钥生成/Nonce/IV

嵌入式TRNG：

- STM32 RNG外设：HAL_RNG_GenerateRandomNumber()
- Linux：/dev/urandom (内核熵池)
- 启动时混合多种熵源(ADC噪声+时钟抖动+MAC地址)

Q67: TLS 1.3的改进?

秒懂： TLS 1.3简化了握手(1-RTT甚至0-RTT)、移除了不安全的算法(RC4/SHA-1等)、握手过程加密(防止中间人窥探)。嵌入式新项目应直接使用TLS 1.3。

TLS 1.3相比1.2的重大改进：

TLS 1.2：	TLS 1.3：
2-RTT握手	1-RTT握手(0-RTT恢复)
支持旧算法(RC4, SHA1)	去除不安全算法
RSA密钥交换(无前向保密)	仅ECDHE(完美前向保密)
ChangeCipherSpec	去除(简化)

嵌入式影响：

- 握手更快(省电/省流量)
- 代码更小(去除旧算法)
- 更安全(密钥泄露不影响历史会话)

mbedtls 3.x 支持TLS 1.3

Q68: 什么是前向保密(Perfect Forward Secrecy)?

秒懂： 前向保密：即使服务器私钥将来泄露，过去的通信仍然安全。原理：每次会话用临时密钥对(DHE/ECDHE)协商会话密钥，密钥不保存。TLS 1.3强制使用前向保密。

即使长期密钥泄露，历史通信仍安全：

无前向保密(RSA密钥交换):

攻击者录制密文 → 未来获取服务器私钥 → 解密所有历史密文!

有前向保密(ECDHE密钥交换):

每次会话生成临时密钥对(ephemeral key)

会话结束后临时密钥销毁

→ 即使私钥泄露也无法解密历史会话

实现: TLS中使用ECDHE-RSA或ECDHE-ECDSA套件

Q69: 嵌入式TCP Socket编程完整示例?

💡 秒懂: 完整TCP客户端/服务器示例: socket→bind→listen→accept→recv/send→close。注意: 处理EINTR、缓冲区管理、超时设置、优雅关闭。嵌入式中加上心跳和重连机制。

面试可能要求手写一个完整的TCP通信程序:

```
// 完整的TCP客户端(含错误处理/超时/重连)
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <errno.h>

int tcp_connect(const char *ip, int port, int timeout_sec) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return -1;

    // 设置非阻塞connect(带超时)
    int flags = fcntl(fd, F_GETFL);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(port)
    };
    inet_pton(AF_INET, ip, &addr.sin_addr);

    int ret = connect(fd, (struct sockaddr *)&addr, sizeof(addr));
    if (ret < 0 && errno == EINPROGRESS) {
        fd_set wfds;
        FD_ZERO(&wfds);
        FD_SET(fd, &wfds);
        struct timeval tv = { .tv_sec = timeout_sec };

        if (select(fd + 1, NULL, &wfds, NULL, &tv) > 0) {
            int err;
            socklen_t len = sizeof(err);
            getsockopt(fd, SOL_SOCKET, SO_ERROR, &err, &len);
            if (err != 0) { close(fd); return -1; }
        } else {
            close(fd); return -1; // 超时
        }
    }
}
```

```

    }
}

// 恢复阻塞模式
fcntl(fd, F_SETFL, flags);
return fd;
}

```

Q70: 嵌入式通信协议设计实战？

💡 秒懂：通信协议设计实战：需求分析(可靠性/速率/方向)→帧格式设计(帧头/类型/长度/数据/CRC)→状态机解析→超时重传→测试验证。嵌入式中协议设计是核心基本功。

完整的私有协议设计示例：

```

// 协议帧定义
#pragma pack(1)
typedef struct {
    uint16_t head;    // 帧头: 0xAA55
    uint8_t  version; // 协议版本
    uint16_t length;  // 数据长度
    uint16_t seq;     // 序列号
    uint8_t  cmd;     // 命令字
    uint8_t  data[0]; // 柔性数组(变长数据)
} Frame;
#pragma pack()

// CRC放在frame末尾: data[length]开始的2字节

// 协议解析状态机
typedef enum {
    STATE_WAIT_HEAD1,
    STATE_WAIT_HEAD2,
    STATE_RECV_HEADER,
    STATE_RECV_DATA,
    STATE_CHECK_CRC
} ParseState;

void protocol_input(uint8_t byte) {
    static ParseState state = STATE_WAIT_HEAD1;
    static uint8_t buf[1024];
    static int pos = 0;

    switch (state) {
    case STATE_WAIT_HEAD1:
        if (byte == 0xAA) { buf[pos++] = byte; state = STATE_WAIT_HEAD2; }
        break;
    case STATE_WAIT_HEAD2:
        if (byte == 0x55) { buf[pos++] = byte; state = STATE_RECV_HEADER; }
        else { pos = 0; state = STATE_WAIT_HEAD1; }
        break;
    // ... 继续解析header和data ...
    }
}

```

```
}
```

本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

🔥 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**

★ 面经高频补充题（来源：GitHub面经仓库/牛客讨论区/大厂真题整理）

Q71: 输入URL到页面显示的全过程？

💡 **秒懂：** URL→DNS解析→TCP三次握手→(TLS握手)→HTTP请求→服务器处理→HTTP响应→浏览器渲染→TCP四次挥手。面试经典题，从网络到应用层层递进回答。

💡 **面试高频** | 牛客/LeetCode面经出现率>80% | 考察全栈网络知识

回答框架（嵌入式设备HTTP请求同理）：

1. DNS解析：URL → IP地址
浏览器缓存 → OS缓存 → 路由器缓存 → ISP DNS → 递归查询
2. TCP三次握手建立连接
SYN → SYN+ACK → ACK
3. TLS握手(HTTPS)：证书验证 → 密钥协商 → 加密通道建立
4. 发送HTTP请求：
GET /index.html HTTP/1.1
Host: www.example.com
5. 服务器处理请求 → 返回HTTP响应(200 OK + HTML)
6. 浏览器渲染：HTML → DOM树 → CSSOM → 渲染树 → 布局 → 绘制
7. TCP四次挥手断开(HTTP/1.0) 或保持连接(HTTP/1.1 Keep-Alive)

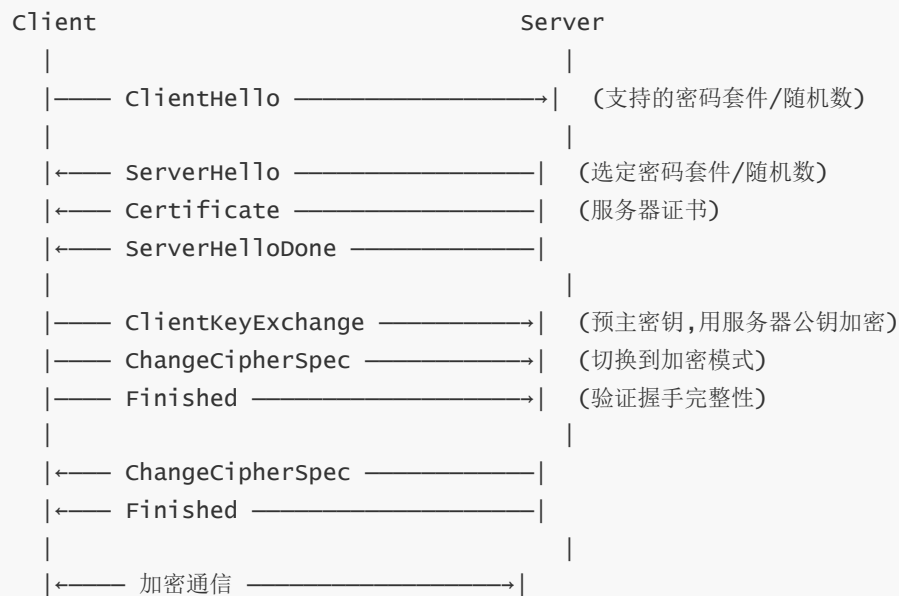
嵌入式关联： 嵌入式设备做HTTP请求(如OTA检查更新)走同样的流程,但通常用轻量级HTTP客户端(如libcurl/mongoose),DNS解析可能直接用硬编码IP跳过。

Q72: HTTPS的TLS握手过程？

💡 **秒懂：** TLS 1.2握手：ClientHello→ServerHello+证书→密钥交换→ChangeCipherSpec→Finished。TLS 1.3简化为1-RTT。理解握手过程是回答HTTPS安全性问题的基础。

💡 **面试高频** | 安全通信必考 | IoT设备TLS是趋势

简化的TLS 1.2握手过程：



嵌入式TLS：mbedtls(轻量) / wolfSSL(适合MCU)

Q73: 粘包和拆包问题？如何解决？

💡 面试高频 | TCP编程必考 | 牛客面经高频 | 嵌入式串口也有类似问题

💡 秒懂：TCP字节流没有消息边界。粘包：read一次读到多条消息混在一起。解决方案：定长协议、长度+数据、分隔符。嵌入式自定义协议必须考虑边界。

解决方案：

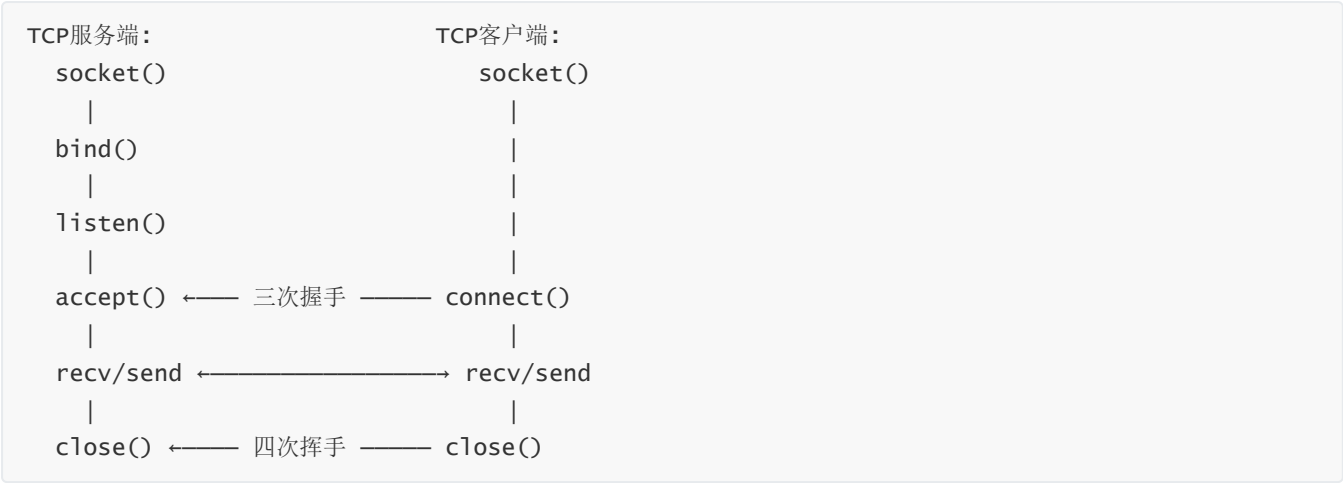
方案	原理	适用场景
固定长度	每个包固定N字节	简单场景/定长数据
分隔符	用\n或特殊字符分隔	文本协议(AT指令)
长度前缀	包头带长度字段	最通用★
自定义协议	帧头+长度+数据+CRC	嵌入式首选

```
// 嵌入式最常用：长度前缀协议
// 帧格式：[0xAA] [0x55] [LEN] [CMD] [DATA...] [CRC]
typedef struct {
    uint8_t header[2]; // 帧头：0xAA 0x55
    uint8_t length;    // 数据长度
    uint8_t cmd;       // 命令字
    uint8_t data[256]; // 数据
    uint8_t crc;       // 校验
} frame_t;
```

Q74: socket编程的基本流程？

💡 秒懂：TCP服务端：socket→bind→listen→accept→read/write→close。客户端：socket→connect→read/write→close。UDP：socket→bind→sendto/recvfrom。核心流程面试必须脱口而出。

💡 面试高频 | 网络编程基础 | 手画流程图



本文档由公众号：**嵌入式校招菌** 原创整理，欢迎关注获取更多嵌入式校招信息

🔴 **嵌入式校招excel** 全年更新中，实习/秋招/春招都包含，纯人工维护，已支持24/25/26届同学毕业，减少招聘信息差，你没听过的公司只要有嵌入式岗位我都会收集，一次付费永久有效，更有嵌入式微信/飞书交流群；用过的同学都说好！

需要的可以关注公众号：**嵌入式校招菌**，对话框回复：**加群**